

Viva Preparation Guide

Microarchitectural & Scaling Analysis of
Context Switching & Scheduling in Multicore Systems
System Software Performance (SSP) — Final Semester Project, 2026

Shikhar Mutta

April 2026

Contents

1	Project Overview Questions	2
2	lmbench & Measurement Methodology	3
3	Cache Hierarchy & Pollution Model	4
4	Core Migration & HyperThreading	5
5	Scheduler Policies	6
6	Process vs Thread & TLB	7
7	Novelty Contributions	8
8	System-Level Terms	9
9	Practical Implications	10
10	Quick Reference Tables	11

1 Project Overview Questions

Q: What is the title and objective of your project?

Answer: Title: *Microarchitectural and Scaling Analysis of Context Switching and Scheduling in Multicore Systems Using lmbench*. The objective is to systematically measure, model, and analyse the latency cost of context switches on an Intel Core i5-1035G1 CPU, studying how CPU cache hierarchy, process count, CPU topology (HyperThreading), and Linux scheduling policies all affect the observed latency.

Q: Why did you choose context switching as your research topic?

Answer: Context switching is the backbone of multitasking in every modern OS, yet most textbooks treat its cost as a single fixed number. In reality, it is a complex function of several hardware and software factors. Understanding this precisely is critical for building low-latency applications like network servers, real-time audio, and high-frequency trading systems.

Q: What is a context switch? Explain step-by-step.

Answer: A context switch is the OS mechanism for switching CPU execution from one process/thread to another:

1. The running process issues a blocking system call (e.g., `read()` on a pipe).
2. The kernel saves its architectural state: general-purpose registers, program counter, stack pointer, FPU state to the task's kernel stack.
3. `schedule()` is called. The Linux CFS scheduler picks the next runnable task from a per-CPU red-black tree in $O(\log N)$ time.
4. For a **process** switch: the page-table base register **CR3** is reloaded, flushing all TLB entries.
5. For a **thread** switch: CR3 is *unchanged* (same address space).
6. The incoming task's register file is restored, and control returns via `SYSRET64`.

Q: What platform did you use and why?

Answer: An Intel Core i5-1035G1 (Ice Lake), a commodity 4-core/8-thread laptop CPU with: L1d=48KB/core, L2=512KB/core, L3=6MB shared, 24GB DDR4 RAM, Ubuntu 22.04 with Linux 6.x kernel. It represents realistic hardware used by students, developers and cloud VMs, making results broadly applicable.

2 lmbench & Measurement Methodology

Q: What is lmbench? What does lat_ctx do?

Answer: lmbench is a portable Unix microbenchmark suite developed by McVoy & Staelin (1996). The lat_ctx tool measures context-switch latency via a **pipe ping-pong**: N processes are chained in a ring. Each process reads 1 byte (blocks $\rightarrow$ context switch occurs), then writes 1 byte to wake the next. The measured round-trip time divided by N gives per-switch latency.

Q: What is the -s flag in lat_ctx?

Answer: The -s <kb> flag tells each process to *touch* (read/write) kb kilobytes of memory before passing the token. This simulates a **cache-polluted** context switch — the cache is dirtied before the switch, forcing the incoming task to reload its data from slower memory levels.

Q: What is RDTSC and why did you use it?

Answer: RDTSC (Read Time-Stamp Counter) is an x86 assembly instruction that reads the CPU's monotonically-increasing cycle counter. We use it to time individual context switches with nanosecond-level resolution. We pair it with CPUID for serialisation, ensuring all preceding instructions retire before the timestamp is taken. This gives us fine-grained, per-iteration distributions, not just means.

Q: Why did you discard the first 200 warm-up iterations?

Answer: The first few iterations suffer from cold-start effects: instruction cache misses, branch predictor warmup, OS page faults on newly allocated working-set buffers, and JIT compilation (if any). Discarding warmup ensures we measure the steady-state behaviour of the benchmark, not the startup overhead.

Q: What is the working-set? What does working-set size mean?

Answer: The **working set** is the set of memory pages (and cache lines) that a process actively accesses during a given time interval. In our benchmarks, it is a buffer of size W KB that the process touches (64-byte stride) before each context switch. Larger working sets evict more of the cache, forcing the incoming task to reload data from slower memory levels.

3 Cache Hierarchy & Pollution Model

Q: What is cache pollution in the context of context switching?

Answer: Before a context switch, the outgoing process may have filled the CPU caches with its own data (its working set). After the switch, the incoming process's data is no longer in cache (it was evicted). The incoming process must re-fetch its working set from slower memory. This cache warming cost is **cache pollution** — the outgoing task's data polluted the cache, slowing down the incoming task.

Q: Explain the three-step staircase latency you observed.

Answer: We observed distinct latency jumps at the cache boundaries:

- **L1 region (0–48 KB):** $\approx 2.8 \mu\text{s}$. Data fits in L1d; misses are rare.
- **L2 region (48–512 KB):** rises to $\approx 5.6 \mu\text{s}$. Each switch evicts $\approx \text{WS}$ KB from L1; must fetch from L2 (14 cycles/access).
- **L3 region (512 KB–6 MB):** $\approx 9.2 \mu\text{s}$. L2 also polluted; L3 ring bus adds contention.
- **DRAM (>6 MB):** $\approx 22.4 \mu\text{s}$, p99=35.8 μs . Full DRAM fetch (200+ cycles) plus high variance from DRAM row-buffer conflicts.

Q: What is the piecewise linear model you derived?

Answer: We fit the empirical means to:

$$\hat{L}(w) = \begin{cases} 2.8 & w \leq 48 \\ 2.8 + 5.3 \log_2(w/48) & 48 < w \leq 512 \\ 8.9 + 4.7 \log_2(w/512) & 512 < w \leq 6144 \\ 20 + 0.3 \log_2(w/6144) & w > 6144 \end{cases}$$

where w is working-set size (KB) and $\hat{L}$ is latency (μs). The model achieves $R^2 = 0.97$, meaning it explains 97% of the observed variance. This lets a system designer predict

context-switch cost for any working-set size without re-running benchmarks.

Q: What is a cache line? What is stride?

Answer: A **cache line** is the minimum unit of data transferred between cache levels and main memory, typically 64 bytes on x86. A **stride** is the step between successive memory accesses. We use a 64-byte stride to touch every cache line exactly once in the working-set buffer, ensuring all lines are loaded into cache without artificial spatial locality.

4 Core Migration & HyperThreading

Q: What is HyperThreading (SMT)?

Answer: HyperThreading (Intel's name for Simultaneous Multi-Threading, SMT) allows one physical core to expose 2 logical CPUs to the OS. Both logical CPUs share the physical core's execution units, L1d cache, and L2 cache. The OS schedules separate threads on each logical CPU simultaneously. This improves throughput but the shared resources can create contention.

Q: What is the HT-sibling vs cross-core migration penalty?

Answer: HT-siblings (same physical core): both processes share L1d and L2, so a context switch between them may find the incoming task's data still warm in cache. Mean latency: $2.4 \mu\text{s}$.

Cross-core: processes are on *different* physical cores with *separate* L1/L2 caches. The incoming task must reload its data from the shared L3 or DRAM. Mean latency: $4.9 \mu\text{s}$ — a **2.0×** penalty.

This is caused by: (1) L1/L2 cache warming needed from L3; (2) cache-coherence snoop transactions transferring modified lines between cores; (3) full TLB flush on CR3 reload.

Q: What is sched_setaffinity? Why did you use it?

Answer: `sched_setaffinity(2)` is a Linux system call that pins a process to a specific set of logical CPUs (CPU affinity mask). We used it to *control* which physical core each process runs on, isolating the HT-sibling vs. cross-core effect from the OS scheduler's autonomous migration decisions.

Q: What is the CPU load balancer? How does it affect context switch cost?

Answer: The Linux CFS load balancer periodically redistributes processes across CPU cores to equalise CPU utilisation. This is good for throughput fairness, but it can *migrate* latency-sensitive processes to a cross-core destination, incurring a $2\times$ latency penalty even when that process has a warm cache on its current core.

5 Scheduler Policies

Q: What is the Linux CFS scheduler?

Answer: CFS (Completely Fair Scheduler) is the default Linux scheduler since kernel 2.6.23. It models CPU time as a virtual clock (**vruntime**) and stores all runnable tasks in a per-CPU **red-black tree** sorted by vruntime. Task selection is $O(\log N)$. CFS aims for fairness: each task gets a proportional share of CPU time, but this fairness introduces scheduling overhead.

Q: What is **SCHED_FIFO**? How is it different from CFS?

Answer: **SCHED_FIFO** is a Linux real-time scheduling class. Key differences from CFS:

- Bypasses the CFS red-black tree entirely, served by an $O(1)$ bitmap-indexed run-queue.
- Priority-ordered: the highest-priority runnable RT task runs until it blocks (no time slicing).
- Our measurement: mean $2.1\ \mu\text{s}$ vs CFS's $3.4\ \mu\text{s}$ — 38% faster.
- p99: $3.4\ \mu\text{s}$ vs CFS's $6.8\ \mu\text{s}$ — 50% lower tail latency.
- Trade-off: a high-priority FIFO task can starve all other processes — sacrifices fairness for latency.

Q: What is **SCHED_IDLE** and why was it slowest?

Answer: **SCHED_IDLE** is a scheduling class below CFS-nice-19. Processes in this class run only when no other non-idle task is runnable. In our benchmark, the kernel allows the CPU to enter a sleep state between ping-pong iterations, greatly inflating the measured time. Mean latency was $8.7\ \mu\text{s}$ — $2.6\times$ slower than FIFO.

Q: What is `SCHED_BATCH`? How does it differ from `SCHED_OTHER`?

Answer: `SCHED_BATCH` is a variant of CFS designed for throughput-oriented workloads. It applies a scheduling penalty to I/O-woken tasks (they are not immediately preemptive), reducing the number of context switches per second. This makes it less reactive than `SCHED_OTHER` (CFS), and we measured slightly higher mean latency ($4.1\ \mu\text{s}$) due to reduced scheduling aggressiveness.

Q: What does `CAP_SYS_NICE` mean?

Answer: It is a Linux capability (a privilege level below full root) that allows a process to raise its scheduling priority to real-time levels (set RT policies like `SCHED_FIFO`/`SCHED_RR` and negative nice values). Without this capability, `sched_setscheduler(2)` will return `EPERM`.

6 Process vs Thread & TLB

Q: What is a TLB and why does it matter for context switches?

Answer: The TLB (Translation Lookaside Buffer) is a small, fast hardware cache storing recent virtual-to-physical address translations. A process switch requires loading a new page table (reloading CR3), which on most x86 CPUs **flushes all non-global TLB entries**. Subsequent memory accesses incur TLB misses until the TLB is repopulated from the new page table — adding significant latency.

Q: Why are thread switches faster than process switches?

Answer: Threads within the same process share the same **virtual address space** and therefore the same page table. A thread switch does *not* reload CR3, so the TLB entries remain valid. We measured the TLB-flush overhead at $1.3 \pm 0.3\ \mu\text{s}$ by comparing process vs. thread switch latency under identical conditions.

Q: What is CR3?

Answer: CR3 is an x86 control register that holds the physical address of the current process's page table base (PML4 table on x86-64). When the OS performs a process switch, it writes the new process's page table address into CR3. This immediately invalidates all non-global TLB entries, forcing the CPU to re-walk the page table for subsequent memory accesses.

Q: What is PCID (Process Context Identifiers)?

Answer: PCID is an x86 feature allowing the TLB to tag entries with a process ID (up to 4096 IDs). With PCID, CR3 reload does *not* immediately flush TLB entries; instead, old entries are preserved and can be restored when that process runs again. Our Ice Lake CPU supports PCID, but we still see a measurable TLB-flush penalty because when more than 4096 address spaces are active, PCID tags are exhausted and a full flush is required.

7 Novelty Contributions

Q: What are the five novel contributions of your project?**Answer:**

1. **Piecewise Cache Pollution Model:** a continuous $\hat{L}(w)$ function mapping working-set size to expected context-switch latency, achieving $R^2 = 0.97$.
2. **HyperThread-Aware Core Migration Penalty:** explicitly separating HT-sibling vs. cross-physical-core scenarios, showing a $2.0\times$ penalty.
3. **Scheduler Fairness vs. Latency Trade-off:** quantifying the latency cost of each of the 5 Linux scheduling classes under identical conditions.
4. **Quantified TLB-Flush Cost:** isolating the exact TLB invalidation overhead to $1.3 \pm 0.3 \mu s$ by comparing process vs. thread switches.
5. **Empirical Cache Miss Tracking:** using `perf_event_open` hardware performance counters to directly count L1d and LLC misses per context switch — bypassing theoretical inference.

Q: What is `perf_event_open`? Why is it significant?

Answer: `perf_event_open` is a Linux system call that programs the CPU's Hardware Performance Monitoring Unit (PMU) to count specific hardware events: cache misses, branch mispredictions, TLB misses, retired instructions, etc. It is significant because it moves from *indirect* latency-based inference ("latency went up, so the cache must have missed") to *direct empirical measurement* ("exactly 8,192 L1d misses occurred during this context switch"). This substantially strengthens the scientific validity of the cache pollu-

tion model.

8 System-Level Terms

Q: What is a pipe? How does lmbench use it for context switch measurement?

Answer: A pipe is a unidirectional inter-process communication (IPC) channel provided by the OS kernel. `read()` on an empty pipe **blocks** the process (triggers a context switch). `write()` on a pipe wakes the blocked reader. lmbench chains N processes: each `read()`s from one pipe (blocks $\rightarrow$ switch), then `write()`s to the next pipe (wakes next process). This creates a precisely controlled context-switch chain.

Q: What is the Completely Fair Scheduler's red-black tree?

Answer: CFS stores all runnable tasks in a **red-black tree** (a self-balancing binary search tree) sorted by each task's **vruntime** (virtual runtime — total CPU time consumed, normalised by priority). The leftmost node (smallest vruntime) is always the next task to run. Insertion, deletion, and leftmost-lookup are all $O(\log N)$, where N is the number of runnable tasks.

Q: What is vruntime?

Answer: **vruntime** is CFS's per-task virtual clock. It increases while the task runs and is normalised by the task's weight (priority-based). CFS always schedules the task with the smallest vruntime, ensuring all tasks accumulate CPU time at an equal rate ("fair" scheduling). Higher-priority tasks have a lower weight divisor, causing their vruntime to grow more slowly, giving them more CPU time.

Q: What is the scheduling latency / timeslice in CFS?

Answer: CFS uses a concept called **targeted scheduling latency** (typically 6ms by default): the period within which every runnable task should get at least one CPU turn. With N tasks, each gets a timeslice of $\text{latency}/N$, with a floor of 0.75ms (minimum granularity). As N grows, timeslices shrink and context switch frequency increases.

Q: What is a syscall overhead? What is VDSO?

Answer: A system call (syscall) is the mechanism for user-space programs to request kernel services, involving a CPU privilege level transition (ring 3 $\rightarrow$ ring 0 $\rightarrow$ ring 3), which costs $\approx 72\text{--}310$ ns on our platform. **VDSO** (Virtual Dynamic Shared Object) is a kernel trick that maps certain frequently-called syscalls (`gettimeofday`, `clock_gettime`) into user-space, avoiding the ring transition entirely. These run in $\approx 16\text{--}18$ ns.

9 Practical Implications

Q: What are your three key recommendations for latency-sensitive applications?

Answer:

1. **Pin threads to HT-sibling pairs** using `taskset` / `sched_setaffinity`. This ensures context switches stay within the same physical core's L1/L2 cache, avoiding the $2\times$ cross-core penalty.
2. **Use `SCHED_FIFO`** with a modest RT priority. This bypasses the CFS red-black tree, reducing mean latency by 38% and p99 by 50%.
3. **Keep per-thread working sets below L2 (512 KB)**. This keeps data in L2 cache even after a context switch, avoiding the sharp latency increase at the L2/L3 boundary.

Q: What are the limitations of your study?

Answer:

- **Single-socket:** No NUMA effects. Multi-socket servers add ≈ 100 ns per NUMA hop.
- **SMT contention not modelled:** When both HT siblings are under heavy load simultaneously, their shared L1/L2 capacity is halved, potentially increasing HT-sibling switch cost.
- **Security mitigations not isolated:** Spectre/Meltdown mitigations (PTI, IBRS) add overhead to CR3 reload and syscalls; we measure with default mitigations enabled.
- **Synthetic workload:** Real applications may not touch memory with a perfect 64-byte stride; access patterns affect cache-hit rates differently.

10 Quick Reference Tables

Summary of All Key Results

Experiment	Scenario	Mean (μs)	p99 (μs)
lmbench baseline	2 procs, WS=0	3.2	—
lmbench scaling	32 procs, WS=0	8.7	—
Cache pollution	DRAM WS	22.4	35.8
Core migration	HT-siblings	2.4	3.8
Core migration	Cross-core	4.9	7.2
Scheduler	SCHED_FIFO	2.1	3.4
Scheduler	SCHED_OTHER	3.4	6.8
Scheduler	SCHED_IDLE	8.7	18.2
Proc vs Thread	Process switch	4.7	—
Proc vs Thread	Thread switch	3.4	—
Syscall baseline	getpid()	0.072	—

Intel Core i5-1035G1 Memory Hierarchy

Level	Size	Latency (cycles)	Shared?
L1d	48 KB/core	$\approx 4-5$	Per core
L2	512 KB/core	≈ 14	Per core
L3	6 MB	≈ 50	All cores
DRAM	24 GB	$\approx 200+$	All cores

Likely Viva Traps — Be Careful:

- “Why is thread switch faster?” — CR3 is *not* reloaded, so TLB is *not* flushed.
- “What does the `-s` flag do?” — it sets working-set size in KB, not processes.
- “What is $R^2 = 0.97$?” — coefficient of determination; model explains 97% of variance.
- “Why use RDTSC and not `clock_gettime`?” — RDTSC is per-cycle resolution; `clock_gettime` calls VDSO with ~ 16 ns overhead, insufficient for sub- μs measurements.
- “What is the difference between lmbench and your custom benchmarks?” — lmbench gives only mean across all iterations; custom benchmarks give p50/p95/p99 per-iteration distributions.